How QA Copilot is built, and why it is built this way

A plain-language overview of the technology behind QA Copilot, an AI-assisted website quality-assurance tool. It is written for managers and leadership: what each part does, why it was chosen, and what it means for cost, security and growth.

1 Summary at a glance

What QA Copilot does: it opens a client's website in a real browser, the same way a visitor would, and checks it automatically for accessibility, security, SEO, broken links, speed, mobile layout and design quality. It compares what it finds with the approved Figma design, brand guidelines and approved text. Every problem comes with proof (a screenshot, the exact element, the measured value) and a plain-language explanation of how to fix it.

7

automated checks

Accessibility, security, SEO, links, performance, mobile layout, design quality, plus design and content comparison.

~16 min

for a 17-page site

Measured on the Ascendion test (24 Oct 2026), checking every page on desktop, tablet and mobile.

$0.04–0.70

AI cost per scan

Depending on the AI model chosen. All the main software is open source, with no licence fees.

547

automated tests

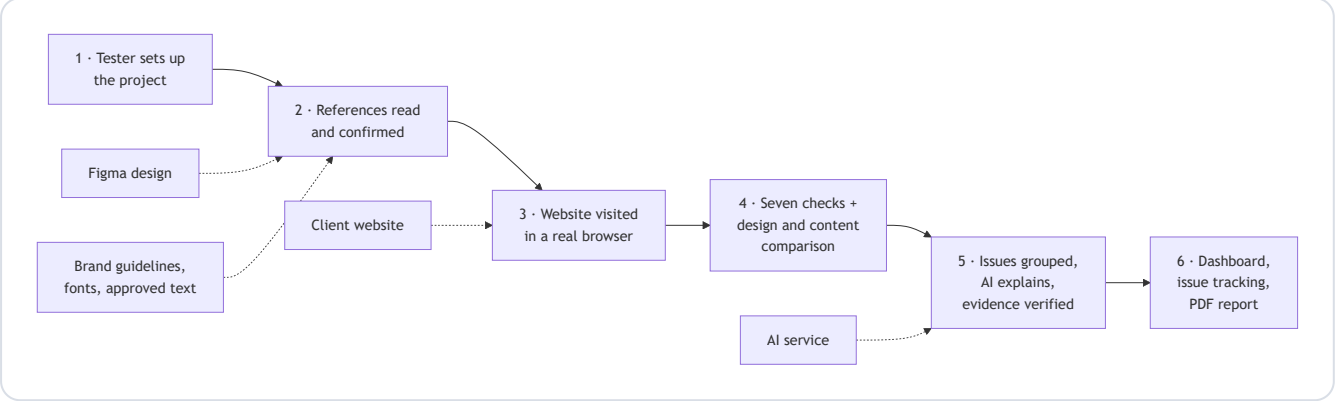
Run on every change to keep the product reliable.

The stack in one sentence: a modern web application (Next.js and React) and a separate processing engine (Node.js with an automated Chrome browser), both written in TypeScript, sharing one PostgreSQL database. It connects to Figma for designs and to an AI provider for explanations.

2 How it works, from start to finish

The process has six steps. The tester only does steps 1, 2 and 6; everything in between is automatic.

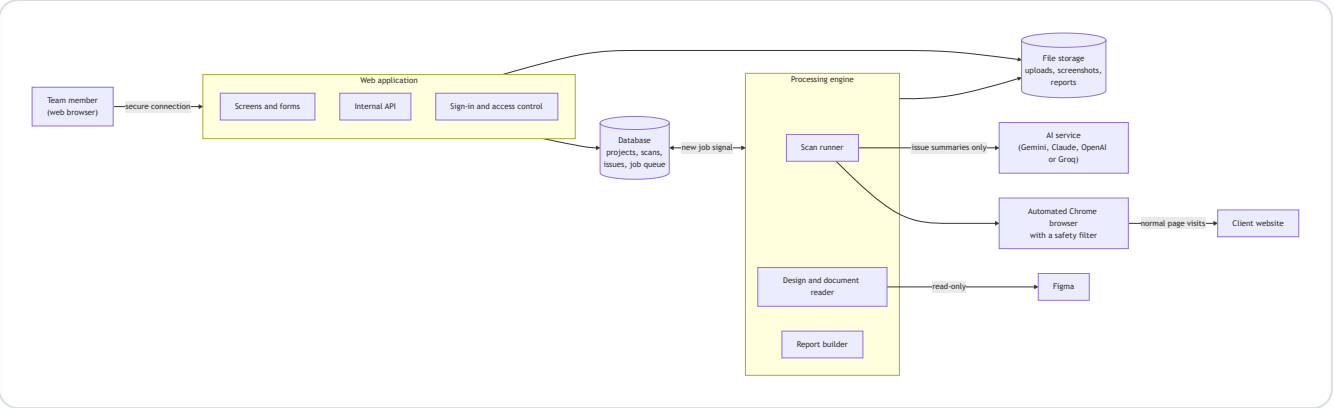
1. **Set up a project.** Enter the website address and add the references: Figma design link, brand guidelines, font files, approved text documents and important page links.
2. **Confirm the references.** The tool reads the Figma design and documents and turns them into a checklist of design values (fonts, sizes, colours). A person reviews and confirms it, so the tool never checks against wrong values.
3. **Visit the website.** The tool finds the site's pages and opens each one in a real browser at desktop, tablet and mobile sizes, recording screenshots and measurements.
4. **Run the checks.** Seven built-in checkers and two comparisons (design and content) look for problems. Each finding is backed by evidence.
5. **Explain the problems.** Similar findings are grouped into issues. AI writes a short, plain-language explanation, and the tool checks that explanation against the evidence before showing it.
6. **Review and report.** The team sees prioritised issues with evidence and fix guides, tracks them across repeated scans, and downloads a PDF report.



> Example: on the Ascendion test the tool visited 17 pages, found that one approved section was missing from an article and that 8 site files were broken on every page, and explained each finding in plain language.

3 System architecture overview

The system has two main parts that share one database. A useful comparison is a restaurant: the **web application** is the front of house that takes orders and serves results; the **processing engine** is the kitchen that does the heavy work; the **database** is the order book both of them read.



- **Why two parts:** checking a website takes minutes and uses a full browser. Keeping that work in a separate engine means the screens stay fast, and a problem on one client website can never crash the application the team is using.
- **One database for everything:** it stores the data and also works as the to-do list for the engine. This removes the need for an extra queueing server.
- **Outside services are limited to two:** Figma (read-only, for designs) and an AI service (for explanations). The AI only receives short issue summaries, never whole websites, files or screenshots.

4 The technology stack, layer by layer

Each layer below shows what it does in plain words, the technology used, and why it was chosen. The reasons come from the project's design documents.

LAYER	WHAT IT DOES	TECHNOLOGY	WHY THIS CHOICE	COMMON ALTERNATIVES
Frontend	The screens the team uses: projects, scan progress, issues, reports	`Next.js 16, React 19, Tailwind CSS 4`	Industry-standard, well supported; screens and API live in one application, so there is less to run	Angular, Vue, separate front and back ends
Backend / API	Receives requests, checks permissions, validates input, queues work	`Next.js API routes, Zod validation`	No second server to operate; every input is checked before use. Long work is never done here, so screens stay fast	Express, Django, Spring
Processing engine	Visits websites, runs the checks, reads designs, builds reports	`Node.js 22, TypeScript`	The same language as the web app, so rules and data definitions are shared, not duplicated. A Python backend was considered and rejected for this reason	Python, Java
Browser automation	Opens pages like a real visitor at three screen sizes, takes screenshots, measures speed, creates PDFs	`Playwright + Chromium`	Sees exactly what visitors see; one tool covers screenshots, measurements and PDF reports	Selenium, Puppeteer

LAYER	WHAT IT DOES	TECHNOLOGY	WHY THIS CHOICE	COMMON ALTERNATIVES
Quality checks	Finds accessibility, design, content and link problems	`axe-core, culori, diff, own rule engines`	axe-core is the industry-standard accessibility engine (WCAG rules); colour comparison uses how humans perceive colour, not exact codes	Lighthouse, WAVE, commercial scanners
Document reading	Reads approved content and guidelines from PDF, Word, Markdown and text; validates font files	`unpdf, mammoth, Markdown parser`	Pure JavaScript, so no extra programs to install or maintain	Commercial document APIs
Database	Stores projects, scans, findings, issues and history; acts as the job queue	`PostgreSQL 18, Drizzle ORM`	Reliable and free; also handles the job queue, so no Redis server is needed. Drizzle was chosen over Prisma as lighter, with readable database changes	MySQL, MongoDB, Redis + BullMQ
File storage	Keeps uploads, screenshots, page snapshots and reports	`Local file storage`	Simple and free; can move to cloud storage later	Amazon S3, Azure Blob
Design integration	Reads design values (fonts, colours, buttons) from Figma	`Figma REST API (read-only)`	Official, supported way to read Figma files; reads only what is needed and skips unchanged files	Manual design specs
AI layer	Writes plain-language explanations of issues	`Provider-neutral layer: Google Gemini (current), Anthropic Claude, OpenAI-compatible, Groq`	Not locked to one vendor; switching is a settings change. Every AI answer is checked against the evidence	Single-vendor integration

LAYER	WHAT IT DOES	TECHNOLOGY	WHY THIS CHOICE	COMMON ALTERNATIVES
Reports	PDF, Markdown and data (JSON) reports	`HTML templates rendered by Chromium`	Reuses the browser the tool already has; professional layout with no extra software	PDF libraries, reporting tools
Security	Sign-in, protection of client data and safe website visits	`Better Auth, encryption, safety filter, rate limits, security headers`	A maintained sign-in library instead of home-made security code (see section 6)	Auth0, Clerk (paid hosted services)
Quality assurance	Automatic tests on every change	`Vitest, Playwright Test, ESLint, Prettier, TypeScript`	Catches mistakes before they reach users; 547 unit tests plus end-to-end tests	Jest, Cypress
Deployment	Packaging to run on a server	`Docker Compose (database, web app, engine)`	One command starts the whole system on any server or cloud. Prepared but not yet run in a Docker environment; today it runs on a local machine	Kubernetes, managed platforms
Cloud services	Hosting	`None required`	Runs on any Windows, Linux or cloud server; the only outside services are Figma and the AI provider	AWS, Azure, Google Cloud

5 Component breakdown

Frontend	A five-step setup wizard, live scan progress, an issue list with evidence and fix guides, design and content comparisons, scan history, issue tracking (assign, resolve, reopen) and report downloads. Built to accessibility standards itself.
Backend and APIs	A versioned internal API (`/api/v1`) for projects, uploads, references, scans, issues and reports. Every request is signed in, permission-checked, input-validated and rate-limited.
Processing engine	Three workers in one service: the **scan runner** (visit pages, run checks, explain issues), the **reference reader** (Figma, guidelines, documents) and the **report builder**. Each scan step records its own status, so nothing is ever reported as done when it was skipped.
Database	PostgreSQL holds all structured data and the job queue. Issues keep a stable identity across scans, so the tool can tell **new**, **resolved** and **reopened** problems apart.
Integrations	**Figma:** read-only design reading, with checks that skip unchanged files to save Figma's limited monthly reads. **AI:** any of four providers; the AI receives only short issue summaries, and its answers are verified against the evidence.
Security	Sign-in with secure sessions; client secrets (Figma and AI keys, site passwords) encrypted; uploaded files checked for type and content; a safety filter that stops scans from reaching internal networks; security headers and rate limits.
Deployment	Three containers (database, web application, processing engine) defined in Docker Compose, with health checks for monitoring.

6 Scalability, performance, security, maintenance and cost

Scalability

Grows by adding processing workers. Each scan runs on its own, and up to 500 pages per scan are supported. More clients means more workers or a bigger server, not a redesign.

Performance

About 16 minutes for a 17-page site on three screen sizes (measured). Heavy work runs in the background, so the screens stay fast while scans run. Repeated design reads are skipped when nothing changed.

Security

Client data stays on our server. Only short issue summaries go to the AI service, and Figma is read-only. Secrets are encrypted, sign-in uses a maintained library, and scans cannot reach private networks.

Maintenance

One programming language end to end, widely used open-source tools, 547 automated tests, clear logs and health checks. A new developer can understand the whole system without learning several languages.

Cost

No licence fees for the core software. Running costs are the server, AI usage (about $0.04–$0.70 per scan depending on the model) and optional Figma seats. A paid AI tier is recommended for client data.

Reliability

Every finding is evidence-backed. Uncertain results are labelled "potential" or "could not be verified" instead of being guessed. If the AI service is down, scans still complete with built-in explanations.

7 Key business advantages

Faster QA cycles

A full website check across three screen sizes in minutes instead of days of manual testing.

Trustworthy results

Every issue shows proof and a fix guide. AI text is fact-checked before it is shown, which reduces false alarms.

Design and brand compliance

Directly compares the live site with the Figma design, brand guidelines and approved text, which most QA tools cannot do.

Low running cost

Open-source foundation, no extra servers such as Redis, and AI costs measured in cents per scan.

No vendor lock-in

The AI provider can be switched in settings, and the system runs on any server or cloud.

Ready to grow

Handles multiple client projects and scales by adding workers; repeated scans show what was fixed and what came back.

Next steps already identified: move client scans to a paid AI tier (data protection), reuse AI explanations for repeated issues, add a backup AI provider, support one design reference per page template, and add a usage and cost dashboard. The separate capacity review covers these in detail.

8 Glossary

API	A defined way for two pieces of software to talk to each other.
Database	Organised storage for the system's data; here PostgreSQL.
Open source	Software whose code is public and free to use, with no licence fees.
Browser automation	Software that controls a real web browser to visit pages automatically.
WCAG	The international standard for website accessibility.
Container (Docker)	A packaged, self-contained unit of software that runs the same way on any server.
AI provider	A company offering AI models through the internet, such as Google (Gemini) or Anthropic (Claude).

Prepared 28 October 2026 from the QA Copilot code, design documents (docs/ARCHITECTURE.md, IMPLEMENTATION_NOTES.md) and measurements from the Ascendion test scans. Versions: Next.js 16.3, React 19.3, Node.js 22, PostgreSQL 18, Playwright 1.63.